

Algorithmic Trading Strategy using Bollinger Band and Stochastic Relative Strength Index

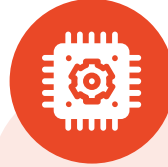
Thu Phuong Nguyen
20202027

Relative Strength Index (RSI)



RSI

- Measures a stock's momentum, indicating speed and size of price changes.
- Helps identify if a stock is overbought or oversold.
- Aids in determining potential entry and exit trading signals.



How to Calculate

$$RSI = \frac{Avg.gain}{Avg.loss}$$

- Define a time period (typically 14 days).
- Calculate average gains and average losses.
- Plot relative strength value on a graph between 0 and 100.

Stochastic Relative Strength Index (StochRSI)



StochRSI

- A min-max normalization of RSI
- More responsive to price changes.
- Also helps identify if a stock is overbought or oversold.



How to Calculate

$$StochRSI = \frac{RSI - \min[RSI]}{\max[RSI] - \min[RSI]}$$

- Define a time period (typically 14 days).
- Calculate RSI on the period
- Plot the indicator on a graph between 0 and 100.

Bollinger Band



Bollinger Band

- Built on top of moving average
- Incorporates recent price volatility
- More adaptive to different market conditions



How to Calculate

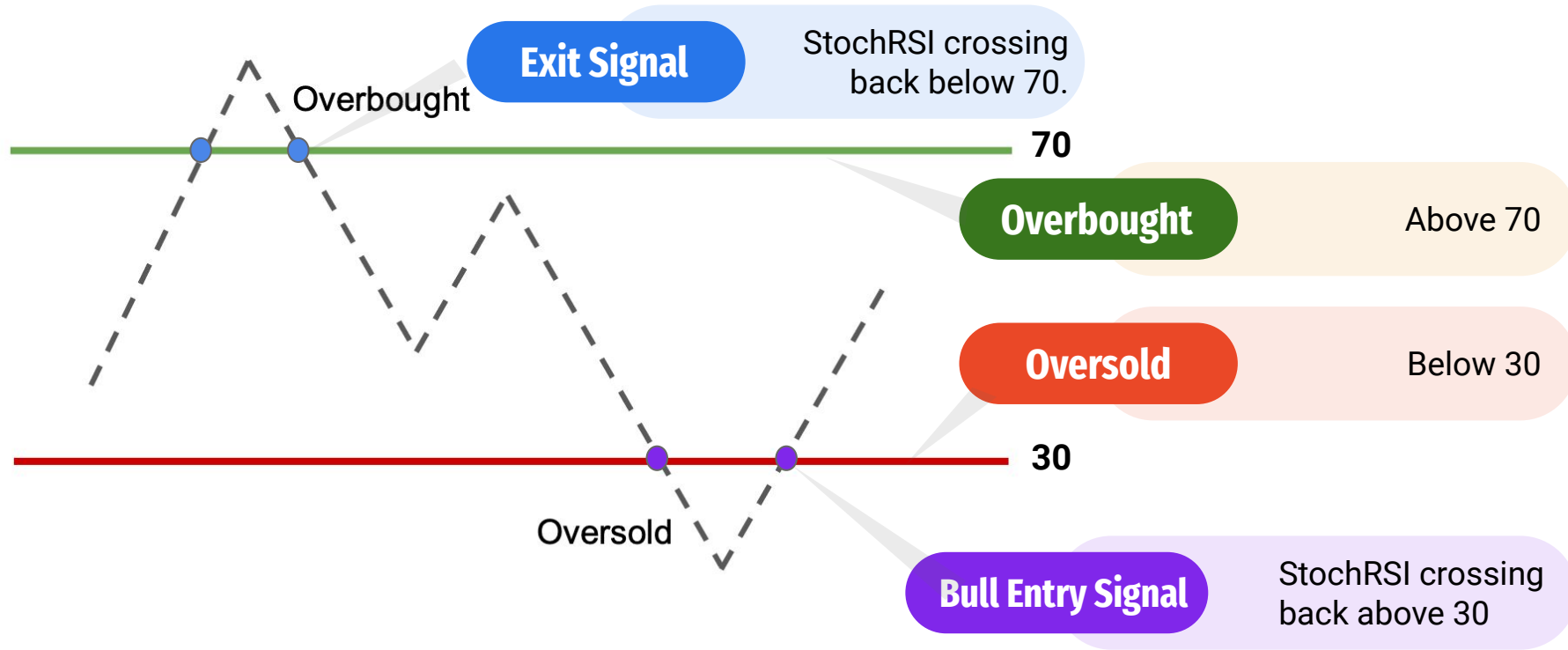
$$BBAND_{Middle} = SMA_{n-periods}$$

$$BBAND_{Upper} = BBAND_{Middle} + (\beta\sigma)$$

$$BBAND_{Lower} = BBAND_{Middle} - (\beta\sigma)$$

Using StochRSI for Trading Signals

The RSI



Using Bollinger Band for Trading Signals

Published on TradingView.com, April 29, 2021 13:18:30 UTC

BITSTAMP:ETHUSD, 1D 2769.68 ▲ +19.31 (+0.7%) O:2751.52 H:2800.89 L:2671.33 C:2769.68



Hyper parameters configuration

```
# Define bot parameters
args = {
    # Stochastic RSI Attributes
    'rsi_period': 14,
    'stochastic_period': 9,

    # Bollinger Band Attributes
    'simple_moving_average_period': 21,
    'bollinger_band_standard_deviation': 2,

    # Buy and Sell Attributes
    'ticker': 'META',
    'stochastic_upper_band': 70,
    'stochastic_lower_band': 30,
    'time_look_back': f'2 M',
    'asset_interval': '15 mins',
    'buy_quantity': 1,
    'show_times': 500,
    'limit_percent': 0.015,
    'initial_cash': 1e4,
}
```

Generating Trading Signals

```
if self.k_fast_array['k_fast'][-1] <= self.stoch_lower and next_price <= self.bb['lower_band'][-1] and self.cash[-1] > next_price * self.buy_quantity:
    self.buy_stock(current_time, next_price)
# If it touches the limit order and stop loss, sell
elif self.k_fast_array['k_fast'][-1] >= self.stoch_upper and next_price >= self.bb['upper_band'][-1] and self.position > 0:
    self.sell_stock(current_time, next_price)
    self.stop_loss.pop()
    self.limit_orders.pop()
elif self.position > 0:
    limit_order_hit = False
    for i in range(len(self.limit_orders)):
        if next_price >= self.limit_orders[i] or next_price <= self.stop_loss[i]:
            print("WOW, stop loss or limit order hit!", current_time, next_price, self.limit_orders[i], self.stop_loss[i])
            self.sell_stock(current_time, next_price)
            self.limit_orders.pop(i)
            self.stop_loss.pop(i)
            limit_order_hit = True
            break
    if not limit_order_hit:
        print("Hold position")
        self.print_values()
        self.orders['time'].append(current_time)
        self.orders['order_limit'].append(None)
        self.orders['order_type'].append('hold')
else:
    print("Hold position")
    self.print_values()
    self.orders['time'].append(current_time)
    self.orders['order_limit'].append(None)
    self.orders['order_type'].append('hold')
```

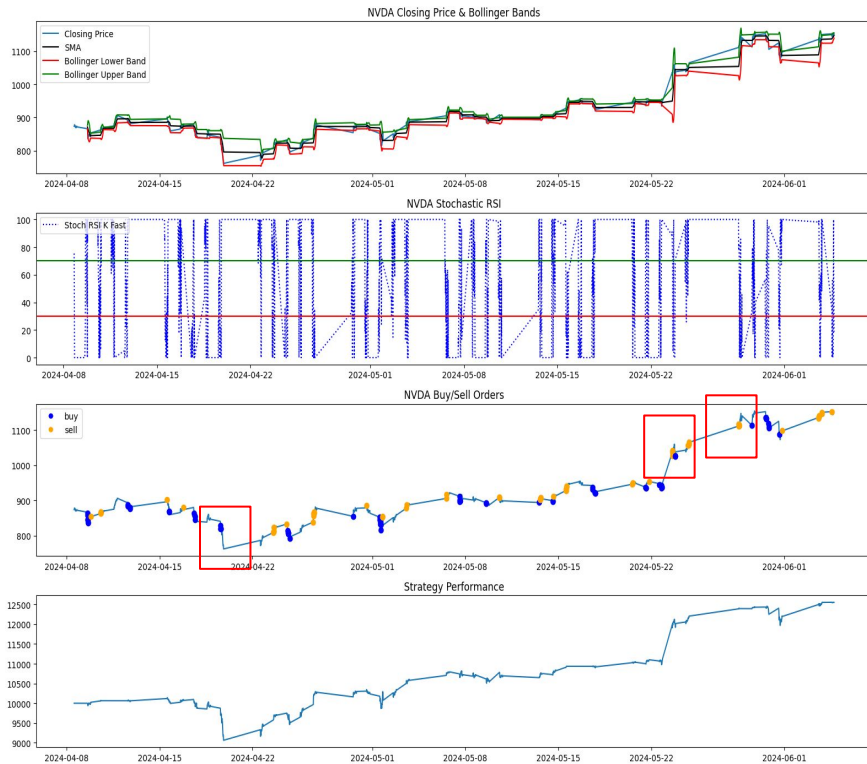
Stop-loss & Profit Taker

```
if self.k_fast_array['k_fast'][-1] <= self.stoch_lower and next_price <= self.bb['lower_band'][-1] and self.cash[-1] > next_price * self.buy_quantity:
    self.buy_stock(current_time, next_price)
# If it touches the limit order and stop loss, sell
elif self.k_fast_array['k_fast'][-1] >= self.stoch_upper and next_price >= self.bb['upper_band'][-1] and self.position > 0:
    self.sell_stock(current_time, next_price)
    self.stop_loss.pop()
    self.limit_orders.pop()
elif self.position > 0:
    limit_order_hit = False
    for i in range(len(self.limit_orders)):
        if next_price >= self.limit_orders[i] or next_price <= self.stop_loss[i]:
            print("WOW, stop loss or limit order hit!", current_time, next_price, self.limit_orders[i], self.stop_loss[i])
            self.sell_stock(current_time, next_price)
            self.limit_orders.pop(i)
            self.stop_loss.pop(i)
            limit_order_hit = True
            break
    if not limit_order_hit:
        print("Hold position")
        self.print_values()
        self.orders['time'].append(current_time)
        self.orders['order_limit'].append(None)
        self.orders['order_type'].append('hold')
else:
    print("Hold position")
    self.print_values()
    self.orders['time'].append(current_time)
    self.orders['order_limit'].append(None)
    self.orders['order_type'].append('hold')
```

Backtesting & Comparison in Bullish market

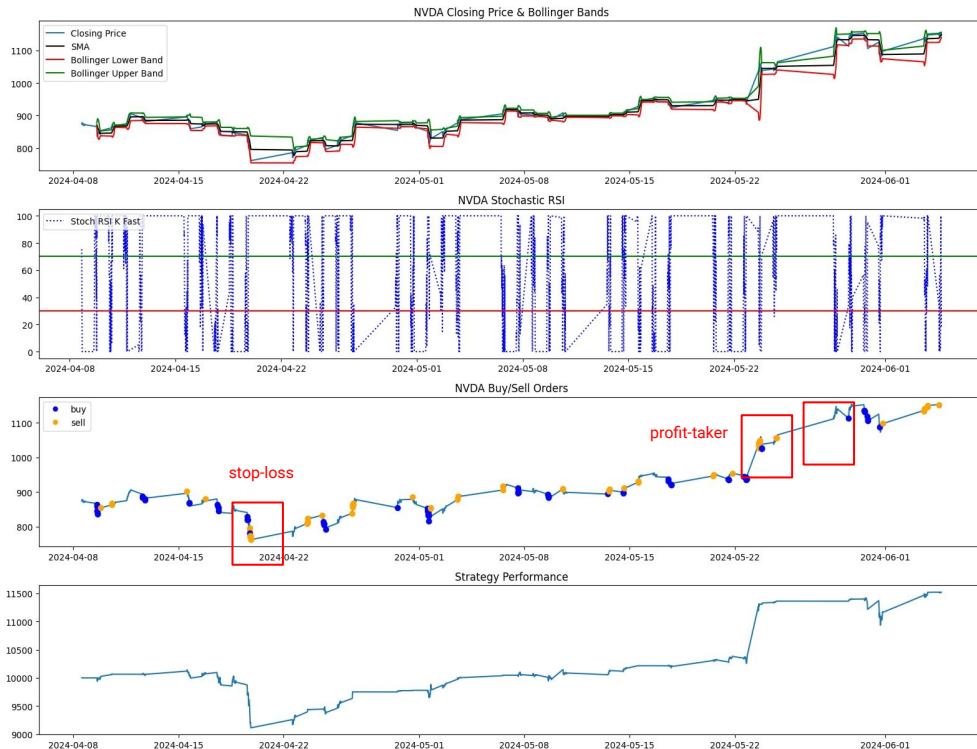
Without stop-loss & profit-taker

With stop-loss & profit-taker



```
sharpe_ratio(net_worth): 0.04590933686508914  
sharpe_ratio(AAPL): 0.025553660515931477
```

>

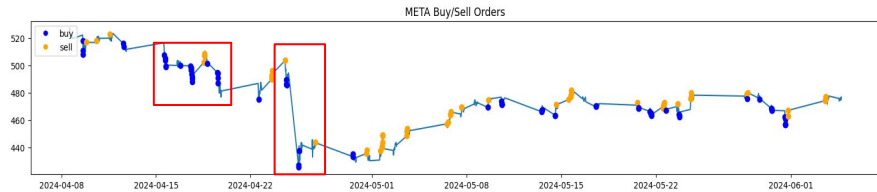
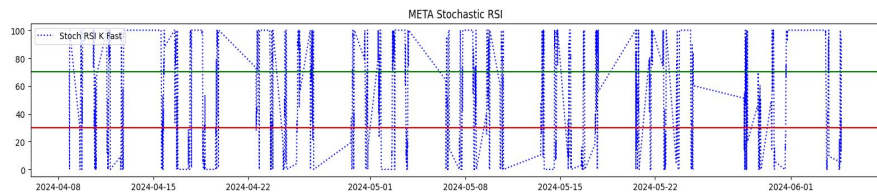
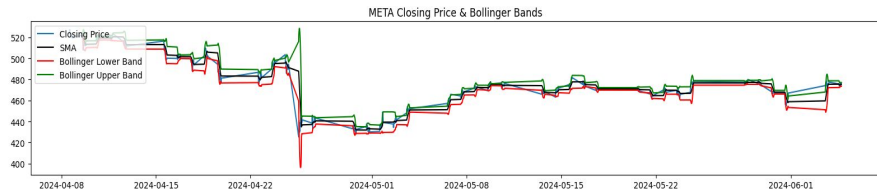


```
sharpe_ratio(net_worth): 0.04013695576177222  
sharpe_ratio(NVDA): 0.04687029245017408
```

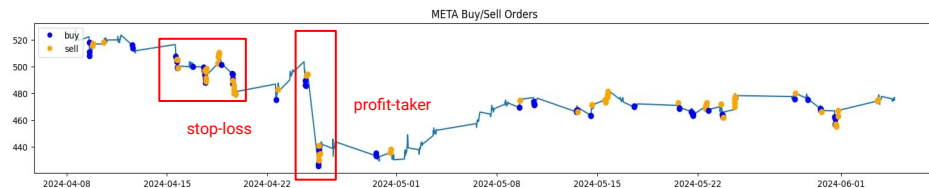
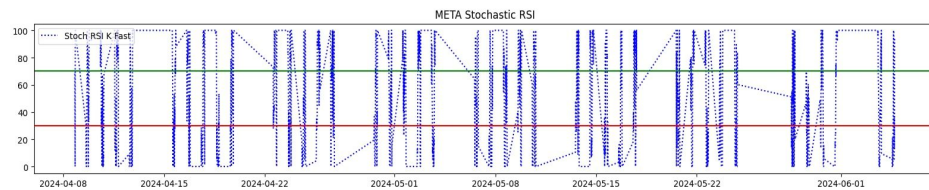
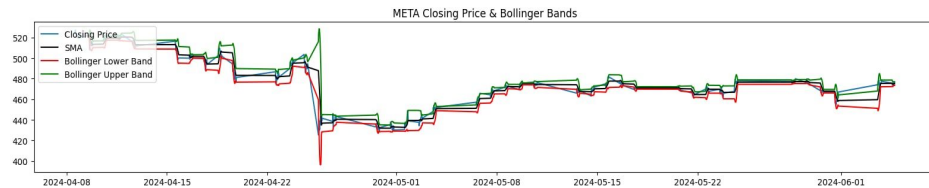
Backtesting & Comparison in Bearish market

Without stop-loss & profit-taker

With stop-loss & profit-taker

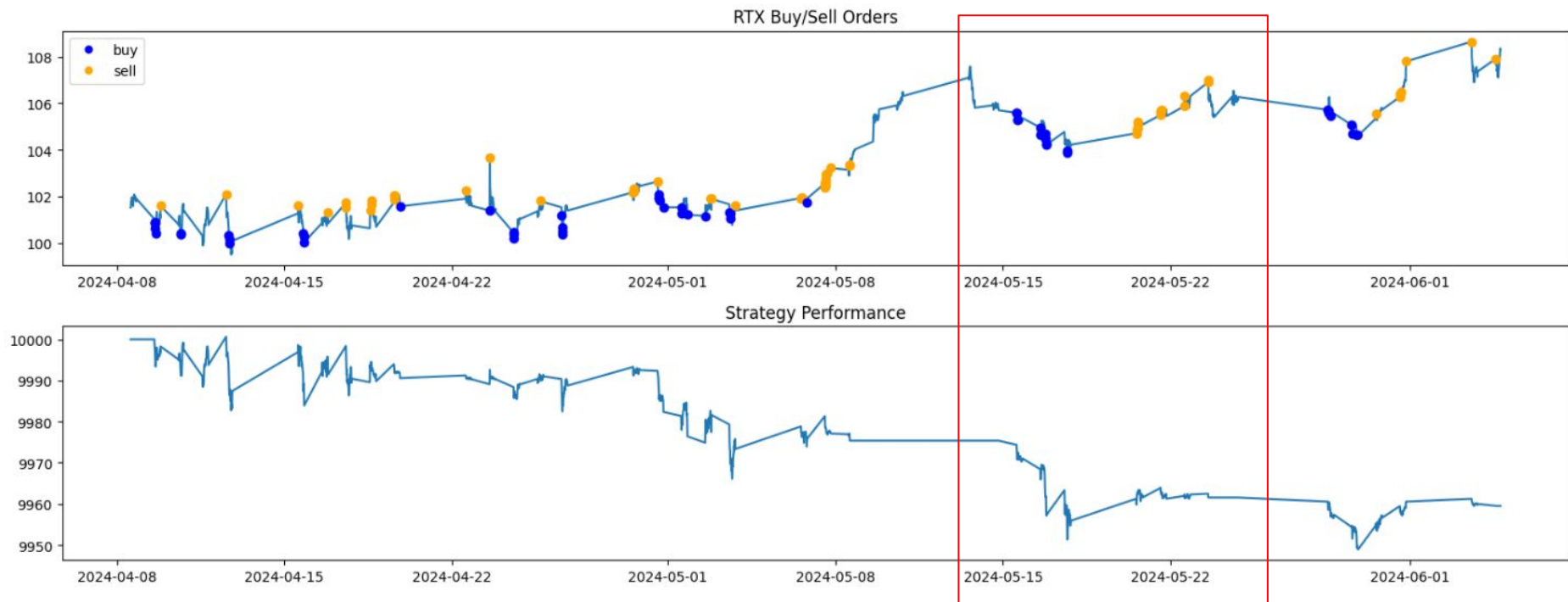


```
sharpe_ratio(net_worth): -0.006520966205531794  
sharpe_ratio(META): -0.013049932228302157
```



```
sharpe_ratio(net_worth): -0.0019076687029308953  
sharpe_ratio(META): -0.013049932228302157
```

Commission Fee



Deploy on Interactive Broker

```
def sell_stock(self, stock, current_time, next_price):
    self.status = 0
    print(f"{current_time} Sell")
    order = MarketOrder("Sell", self.buy_quantity)
    self.ib.placeOrder(stock, order)
    print('Current Price:', next_price, '\nCreated Sell Order:', self.bb['upper_band'][-1], '\nQuantity:', self.buy_quantity, '\nTime:', current_time)
    self.position -= 1
    self.orders['time'].append(current_time)
    self.orders['order_limit'].append(next_price)
    self.orders['order_type'].append('sell')
    self.print_values()

def buy_stock(self, stock, current_time, next_price):
    self.status = 1
    print(f"{current_time} Buy")
    order = MarketOrder("Buy", self.buy_quantity)
    self.ib.placeOrder(stock, order)
    print('Current Price:', next_price, '\nCreated Buy Order:', self.bb['lower_band'][-1], '\nQuantity:', self.buy_quantity, '\nTime:', current_time)
    self.position += 1
    self.orders['time'].append(current_time)
    self.orders['order_limit'].append(next_price)
    self.orders['order_type'].append('buy')
    self.print_values()
```

Deploy on Interactive Broker

MONITOR

Portfolio

Favorites

US Movers

+

Filter ?

P&L

+ PROFILE

+ IMPACT

DAILY

151

Unrealized

220.7

Since prior Close ▾

0.01%

Realized

Margin

+ ACCOUNT

Net Liquidity

1.0M

Excess Liq

1.0M

Maintenance

9.0K SMA

940.9K

DAILY P&L	FIN INSTR	POSITION	MKT VAL	AVG PX	LAST	CHANGE
-1	INTC	12	362	30.69 ¹	30.18	-0.11
6	RTX	12	1,294	106.47 ⁹	107.89	+0.52
0	GOOG	4	697	175.38 ⁸	174.31	-0.11
-11	AMD	7	1,134	165.48 ⁹	161.97	-1.58
-3	AMZN	8	1,424	180.97 ³	178.00	-0.34
-6	MSFT	4	1,648	419.79 ⁸	411.87	-1.65
188	COST	19	15,681	817.04 ⁵	825.50	+10.11
-19	AAPL	45	8,712	192.58 ⁸	193.51	-0.52
-3	META	11	5,249	468.90 ¹	477.09	-0.40
USD CASH			989,366			

